Q1. What are the key limitations of traditional MapReduce that make Spark a preferred choice for modern big data processing?

Traditional MapReduce relies heavily on disk-based processing, where the output of one stage is written to disk before being used by the next stage. This increases execution time and makes processing slower, especially for large datasets. It is not suitable for iterative tasks such as machine learning because data must be repeatedly read from and written to disk. The programming model is also more complex as developers need to write separate Map and Reduce functions. Apache Spark addresses these limitations through in-memory processing, faster execution, easier APIs, support for SQL, machine learning, graph processing, and real-time analytics.

Q2. Explain how Spark uses In-Memory Computing to speed up iterative machine learning algorithms compared to disk-based systems.

Spark uses in-memory computing by storing intermediate data in RAM instead of writing it to disk after every operation. Machine learning algorithms often require multiple iterations over the same dataset. In traditional systems, each iteration involves reading and writing data from disk, which increases processing time. Spark keeps frequently used data in memory, allowing subsequent operations to access it quickly. This significantly reduces I/O overhead and improves performance, making Spark highly efficient for machine learning and data analytics tasks.

Q3. Write a code snippet to remove all duplicate rows from a DataFrame based on a specific set of columns: user_id and transaction_date.

```
clean_df = df.dropDuplicates(
    ["user_id", "transaction_date"]
)


clean_df.show()
```

Q4. Given a DataFrame df_sales, write a query to filter for rows where the region is 'West' and then group by product_category to find the average sale_amount.

```
from pyspark.sql.functions import avg


result = (
    df_sales
    .filter(df_sales.region == "West")
```

```
    .groupBy("product_category")
    .agg(avg("sale_amount").alias("average_sale_amount"))
)

result.show()
```

Q5. What is the difference between .na.drop() and .na.fill()? Provide a code example of filling null values in a status column with the string 'Unknown'.

The .na.drop() function removes rows containing null values, whereas .na.fill() replaces null values with a specified value. When data is important and should not be removed, filling missing values is generally preferred.

```
df = df.na.fill(
    {"status": "Unknown"}
)

df.show()
```

Q6. Write a query to find the total count of records for each city in a DataFrame, but only for cities where the count is greater than 100.

```
from pyspark.sql.functions import count

result = (
    df
    .groupBy("city")
    .agg(count("*").alias("record_count"))
    .filter("record_count > 100")
)

result.show()
```

Q7. How does the immutability of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?

Spark DataFrames are immutable, meaning their contents cannot be changed after creation. Whenever a cleaning operation such as dropping a column, renaming a column, or filtering records is performed, Spark creates a new DataFrame instead of modifying the existing one. This approach ensures fault tolerance and allows Spark to optimize execution plans efficiently. Therefore, data cleaning is performed by creating transformed versions of the original DataFrame.

Q8. Write a Spark command to filter a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

```
from pyspark.sql.functions import col


premium_users = df.filter(
    (col("age") >= 18) &
    (col("age") <= 30) &
    (col("subscription") == "Premium")
)


premium_users.show()
```

Q9. When cleaning a dataset, why is it often better to handle null values before performing mathematical aggregations like sum() or avg()?

Handling null values before performing mathematical aggregations helps ensure accurate and meaningful results. Null values may lead to incorrect calculations, missing outputs, or inconsistencies in analysis. By replacing or removing null values before applying functions such as sum(), average(), minimum(), or maximum(), the quality and reliability of the results are improved. Proper null handling is therefore an important step in data preprocessing.

Q10. Write the code to revise a column named raw_timestamp by casting it to a TimestampType and renaming it to event_time.

```
from pyspark.sql.functions import col
from pyspark.sql.types import TimestampType


updated_df = (
```

```
    df
    .withColumn(
        "raw_timestamp",
        col("raw_timestamp").cast(TimestampType())
    )
    .withColumnRenamed(
        "raw_timestamp",
        "event_time"
    )
)


updated_df.show()
```

Q11. Explain the "Shuffle" process that occurs during a grouping operation. Why is it considered a wide transformation?

Shuffle is the process of redistributing data across different partitions so that records with the same key can be brought together. During operations such as groupBy(), join(), and distinct(), Spark transfers data between executors across the cluster. Since records from multiple partitions are exchanged and reorganized, the operation involves significant network communication and processing overhead. For this reason, grouping operations are classified as wide transformations. Wide transformations are generally more expensive than narrow transformations because they require data movement across partitions.

Q12. Write a code snippet that identifies and removes rows where the email column contains null values OR the username is an empty string.

```
from pyspark.sql.functions import col


clean_df = df.filter(
    col("email").isNotNull()
).filter(
    col("username") != ""
```

```
)

clean_df.show()
```

Q13. How do you use the .agg() function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

```
from pyspark.sql.functions import min, max, avg

statistics = df.agg(
    min("price").alias("minimum_price"),
    max("price").alias("maximum_price"),
    avg("price").alias("average_price")
)

statistics.show()
```

The .agg() function allows multiple aggregation operations to be performed in a single query, making data analysis more efficient.

Q14. In the context of cleaning a dataset, what is the risk of using inferSchema=true when your source data contains messy or inconsistent date formats?

When inferSchema=true is used, Spark automatically determines data types based on the available data. If date formats are inconsistent, Spark may incorrectly interpret values, treat dates as strings, or generate null values during parsing. This can lead to inaccurate analysis, failed transformations, and data quality issues. To avoid such problems, it is often better to define the schema explicitly when working with datasets that contain inconsistent date formats.

Q15. Write a final processing pipeline that filters out duplicates, fills null prices with 0, and groups by store_id to calculate total revenue.

```
from pyspark.sql.functions import sum

final_result = (
```

```
    sales_df
    .dropDuplicates()
    .fillna({"price": 0})
    .groupBy("store_id")
    .agg(
        sum("price").alias("total_revenue")
    )
)

final_result.show()
```

This pipeline first removes duplicate records, then replaces missing price values with zero, and finally groups the data by store ID to calculate the total revenue generated by each store. It demonstrates a complete Spark workflow involving data cleaning, transformation, and aggregation.